

Part 1: The Phantom Bug

Write a program that tosses 50,000,000 random (x, y) points into a 1×1 square to approximate π ($\pi \approx 4 \times \frac{\text{hits}}{\text{total}}$). • Split the loop across 4 native Java threads. • Have all 4 threads update a shared variable: static long totalHits = 0; via totalHits++. • Deliverable: Run it 5 times. Record the value of π . • The Lesson: The code runs fast, but gives $\pi \approx 1.8$ to 2.4 because threads overwrite each other's memory writes (data race)

Run	Pi	Time
1	3.14170	941 ms
2	3.14190	825 ms
3	3.14175	797 ms
4	3.14155	841 ms
5	3.14162	827 ms
6	3.14148	852 ms

```
"D:\Program Files\Java\jdk-21\bin\java.e
Total hits: 39268514
Pi = 3.14148112
Time: 852 ms

Process finished with exit code 0
```

Part 2: The Synchronization Trap

Fix the race condition by adding synchronized to the increment or using AtomicLong.incrementAndGet(). • Deliverable: Measure execution time vs. a plain, single-threaded for loop. • The Lesson: The answer is now accurate (≈ 3.1415), but it runs drastically slower than a single thread because cores spend 95% of their cycles stalling on memory bus locks.

```

"D:\Program Files\Java\jdk-21\bin
=== MULTITHREADED ===
Total hits: 39270518
Pi = 3.14164144
Time: 4233 ms

=== SINGLE-THREADED ===
Total hits: 39264593
Pi = 3.14116744
Time: 2915 ms

```

Version		Pi	Time
Multithreaded + synchronized	4	3.14164	4233 ms
Single-threaded	1	3.14117	2915 ms

The race condition was fixed using the synchronized method.

The multithreaded version with 4 threads calculated $\text{Pi} = 3.14164144$ and took 4233 ms.

The single-threaded version calculated $\text{Pi} = 3.14116744$ and took 2915 ms.

Both results are accurate and close to $\pi \approx 3.1415$. However, the synchronized multithreaded version was slower than the single-threaded version. This happens because the threads must wait when accessing the shared counter. Synchronization creates overhead and reduces the benefit of using multiple threads.

Part 3: OpenMP-Style Reduction

Eliminate shared-state locks. Give each thread a private, local counter (conceptually identical to `#pragma omp parallel for reduction(+:totalHits)`). • Accumulate local hits inside the thread, then add the partial sums together only once at the very end when joining threads. •

Deliverable: Benchmark total execution time for T in $\{1, 2, 4, 8, 16, 32\}$ threads on their machine with 100,000,000 iterations.

```

"D:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program
=== OPENMP-STYLE REDUCTION ===
Total iterations: 100000000

Threads: 1 | Hits: 78533182 | Pi: 3.14132728 | Time: 5495 ms
Threads: 2 | Hits: 78542672 | Pi: 3.14170688 | Time: 2865 ms
Threads: 4 | Hits: 78535620 | Pi: 3.1414248 | Time: 1813 ms
Threads: 8 | Hits: 78541819 | Pi: 3.14167276 | Time: 1879 ms
Threads: 16 | Hits: 78536093 | Pi: 3.14144372 | Time: 1976 ms
Threads: 32 | Hits: 78539331 | Pi: 3.14157324 | Time: 4651 ms

Process finished with exit code 0

```

In Part 3, the shared counter was eliminated. Each thread used its own local counter and the partial results were combined only after all threads finished.

The program was tested with 100,000,000 iterations using 1, 2, 4, 8, 16, and 32 threads.

The execution times were:

1 thread - 5495 ms
 2 threads - 2865 ms
 4 threads - 1813 ms
 8 threads - 1879 ms
 16 threads - 1976 ms
 32 threads - 4651 ms

All calculated Pi values were close to 3.1415. The best measured execution time was 1813 ms with 4 threads. Increasing the number of threads beyond 4 did not improve performance and eventually made the program slower.

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread	Efficiency
1	5495	1.00*	100%
2	2865	1.92*	96.0%

4	1813	3.03*	75.8%
8	1879	2.93*	36.6%
16	1976	2.78*	17.4%
32	4651	1.18*	3.7%

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

An 8-core CPU has 8 physical cores. With 16 threads, multiple threads have to share the same CPU cores and resources. This creates overhead and competition between threads, so 16 threads cannot provide twice the performance of 8 threads.

2. Why was the synchronized version in Part 2 slower than running on one single core?

The synchronized version was slower because all threads shared the same counter and had to wait for each other when updating it. The synchronized lock created significant overhead and reduced parallelism. As a result, the 4-thread version took 4233 ms, while the single-threaded version took only 2915 ms.